STEP-1: EXPLORATORY DATA ANALYSIS

In [ ]:
```
# 1. IMPORTING ALL REQUIRED LIBRARIES
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix, classification_r
# Voting / Stacking / Boosting
from sklearn.ensemble import VotingClassifier, StackingClassifier
from sklearn.ensemble import GradientBoostingClassifier, AdaBoostClassifier
import xgboost as XGBClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
```

In [ ]:
```
# STEP 2: SET DISPLAY OPTIONS
pd.set_option('display.max_columns', None)
```

In [ ]:
```
# STEP 3: LOAD THE DATASET
df = pd.read_csv('Thyroid_Diff.csv')

# Display the dataframe
df.head()
```

Out[ ]:

	Age	Gender	Smoking	Hx Smoking	Hx Radiothreapy	Thyroid Function	Physical Examination	Ader
0	27	F	No	No	No	Euthyroid	Single nodular goiter-left	
1	34	F	No	Yes	No	Euthyroid	Multinodular goiter	
2	30	F	No	No	No	Euthyroid	Single nodular goiter-right	
3	62	F	No	No	No	Euthyroid	Single nodular goiter-right	
4	62	F	No	No	No	Euthyroid	Multinodular goiter	

In [ ]:

```
# STEP 4: BASIC DATA CHECK

print("SHAPE:\n", df.shape)
print("\nINFO:\n")
print(df.info())
print("\nDESCRIBE:\n")
print(df.describe(include='all'))
print("\nCOLUMNS:\n")
print(df.columns)
```

SHAPE:
 (383, 17)

INFO:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 383 entries, 0 to 382
Data columns (total 17 columns):
 #   Column   Non-Null Count Dtype
--- ------   -------------- -----
 0   Age   383 non-null   int64
 1   Gender   383 non-null   object
 2   Smoking   383 non-null   object
 3   Hx Smoking   383 non-null   object
 4   Hx Radiothreapy   383 non-null   object
 5   Thyroid Function   383 non-null   object
 6   Physical Examination 383 non-null   object
 7   Adenopathy   383 non-null   object
 8   Pathology   383 non-null   object
 9   Focality   383 non-null   object
 10 Risk   383 non-null   object
 11 T   383 non-null   object
 12 N   383 non-null   object
 13 M   383 non-null   object
 14 Stage   383 non-null   object
 15 Response   383 non-null   object
 16 Recurred   383 non-null   object
dtypes: int64(1), object(16)
memory usage: 51.0+ KB
None

DESCRIBE:

                Age Gender Smoking Hx Smoking Hx Radiothreapy Thyroid Function
\
count   383.000000   383   383   383   383   383
unique   NaN   2   2   2   2   5
top   NaN   F   No   No   No   Euthyroid
freq   NaN   312   334   355   376   332
mean   40.866841   NaN   NaN   NaN   NaN   NaN
std   15.134494   NaN   NaN   NaN   NaN   NaN
min   15.000000   NaN   NaN   NaN   NaN   NaN
25%   29.000000   NaN   NaN   NaN   NaN   NaN
50%   37.000000   NaN   NaN   NaN   NaN   NaN
75%   51.000000   NaN   NaN   NaN   NaN   NaN
max   82.000000   NaN   NaN   NaN   NaN   NaN

       Physical Examination Adenopathy Pathology   Focality Risk   T   N \
count   383   383   383   383 383 383 383
unique   5   6   4   2   3   7   3
top   Multinodular goiter   No Papillary Uni-Focal Low   T2   N0
freq   140   277   287   247 249 151 268
mean   NaN   NaN   NaN   NaN NaN NaN NaN
std   NaN   NaN   NaN   NaN NaN NaN NaN

```
min                   NaN        NaN        NaN        NaN  NaN  NaN  NaN
25%                   NaN        NaN        NaN        NaN  NaN  NaN  NaN
50%                   NaN        NaN        NaN        NaN  NaN  NaN  NaN
75%                   NaN        NaN        NaN        NaN  NaN  NaN  NaN
max                   NaN        NaN        NaN        NaN  NaN  NaN  NaN

          M Stage   Response Recurred
count   383   383        383      383
unique    2     5          4        2
top      M0     I  Excellent       No
freq    365   333        208      275
mean    NaN   NaN        NaN      NaN
std     NaN   NaN        NaN      NaN
min     NaN   NaN        NaN      NaN
25%     NaN   NaN        NaN      NaN
50%     NaN   NaN        NaN      NaN
75%     NaN   NaN        NaN      NaN
max     NaN   NaN        NaN      NaN

COLUMNS:

Index(['Age', 'Gender', 'Smoking', 'Hx Smoking', 'Hx Radiothreapy',
       'Thyroid Function', 'Physical Examination', 'Adenopathy', 'Pathology',
       'Focality', 'Risk', 'T', 'N', 'M', 'Stage', 'Response', 'Recurred'],
      dtype='object')
```

In [ ]:
```python
# STEP 5: CLEAN COLUMN NAMES
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")
df.columns
```

Out[ ]:
```
Index(['age', 'gender', 'smoking', 'hx_smoking', 'hx_radiothreapy',
       'thyroid_function', 'physical_examination', 'adenopathy', 'pathology',
       'focality', 'risk', 't', 'n', 'm', 'stage', 'response', 'recurred'],
      dtype='object')
```

In [ ]:
```python
# STEP 6: CHECK MISSING VALUES
df.isnull().sum()     # null values
```

Out[ ]:

	0
age	0
gender	0
smoking	0
hx_smoking	0
hx_radiothreapy	0
thyroid_function	0
physical_examination	0
adenopathy	0
pathology	0
focality	0
risk	0
t	0
n	0
m	0
stage	0
response	0
recurred	0

dtype: int64

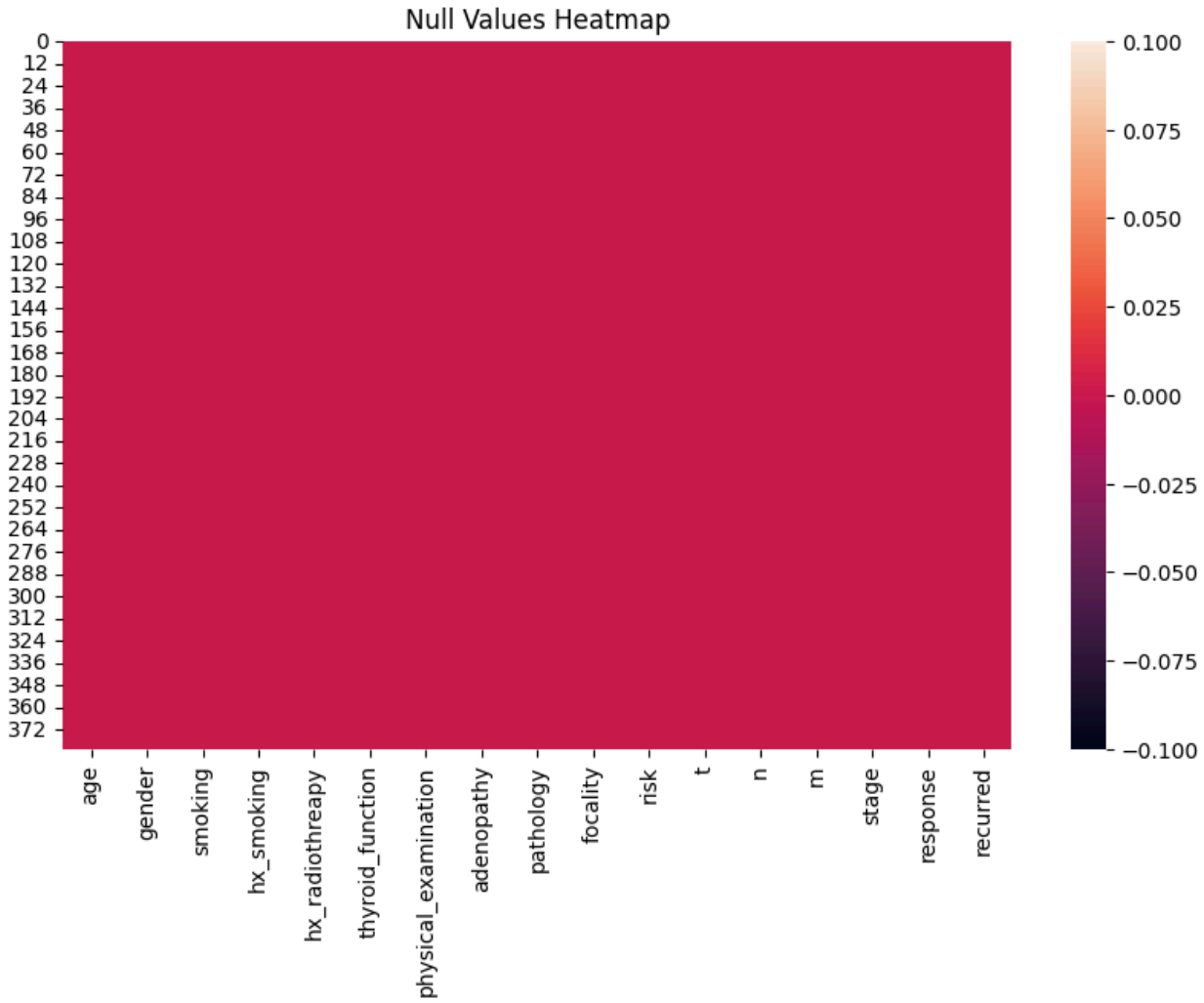
In [ ]:
```
df.notnull().sum()    # not-null values
```

Out[ ]:

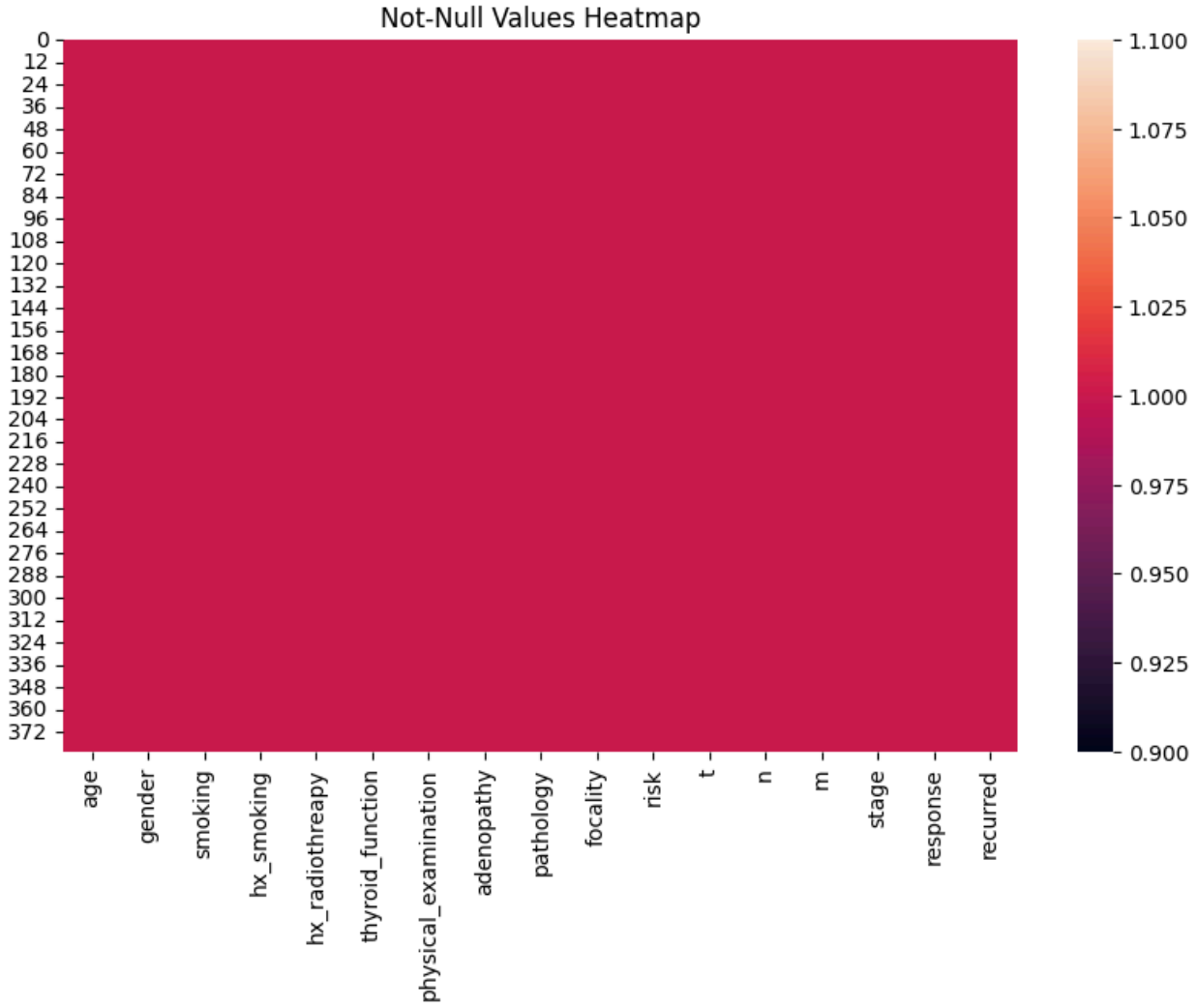
	0
age	383
gender	383
smoking	383
hx_smoking	383
hx_radiothreapy	383
thyroid_function	383
physical_examination	383
adenopathy	383
pathology	383
focality	383
risk	383
t	383
n	383
m	383
stage	383
response	383
recurred	383

dtype: int64

In [ ]:
```
# STEP 7: HEATMAP OF NULL VALUES
plt.figure(figsize=(10, 6))
sns.heatmap(df.isnull(), cbar=True)
plt.title("Null Values Heatmap")
plt.show()
```



In [ ]:
```
plt.figure(figsize=(10, 6))
sns.heatmap(df.notnull(), cbar=True)
plt.title("Not-Null Values Heatmap")
plt.show()
```



In [ ]:
```
# STEP 8: CHECK DUPLICATES
df.duplicated().sum()
```

Out[ ]:
```
np.int64(19)
```

In [ ]:
```
# Drop duplicates
df = df.drop_duplicates()
print("Shape after removing duplicates:", df.shape)
```

```
Shape after removing duplicates: (364, 17)
```

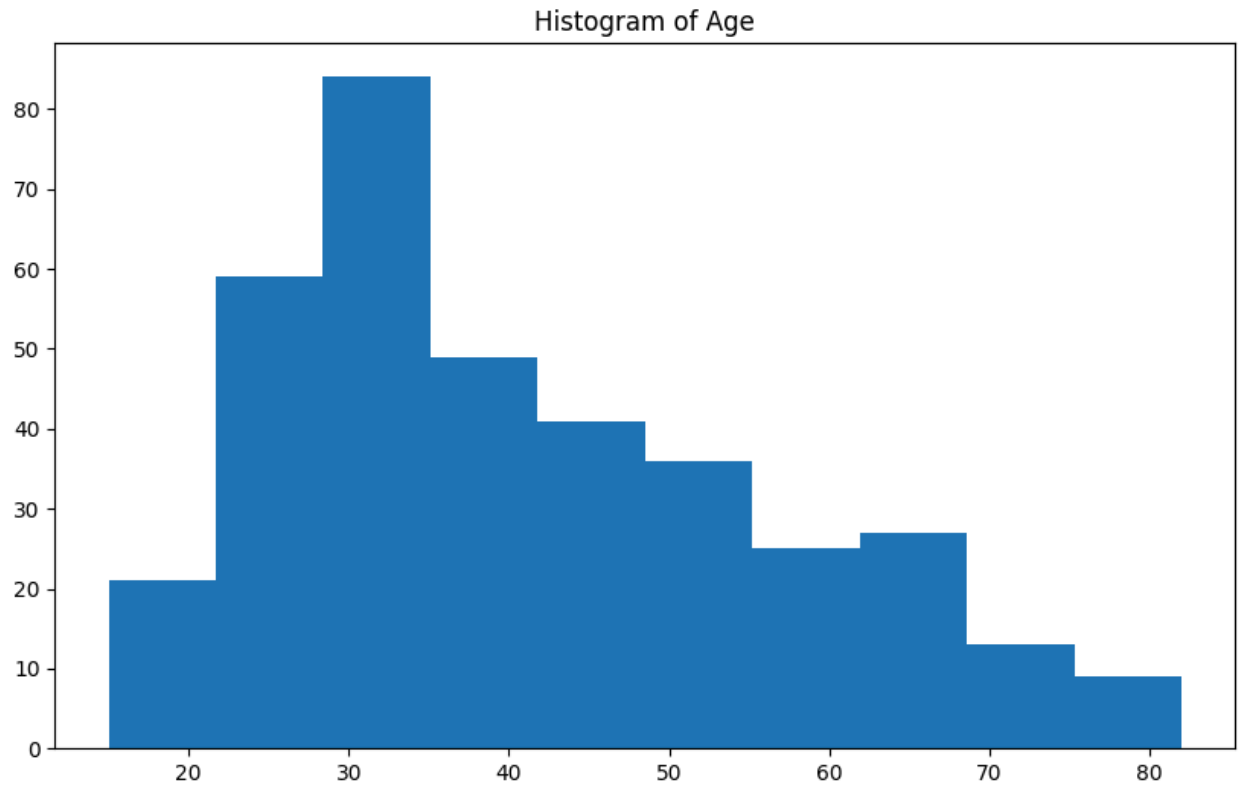
In [ ]:
```
# STEP 9: CHECK & FIX DATA TYPES
df.dtypes
```

Out[ ]:

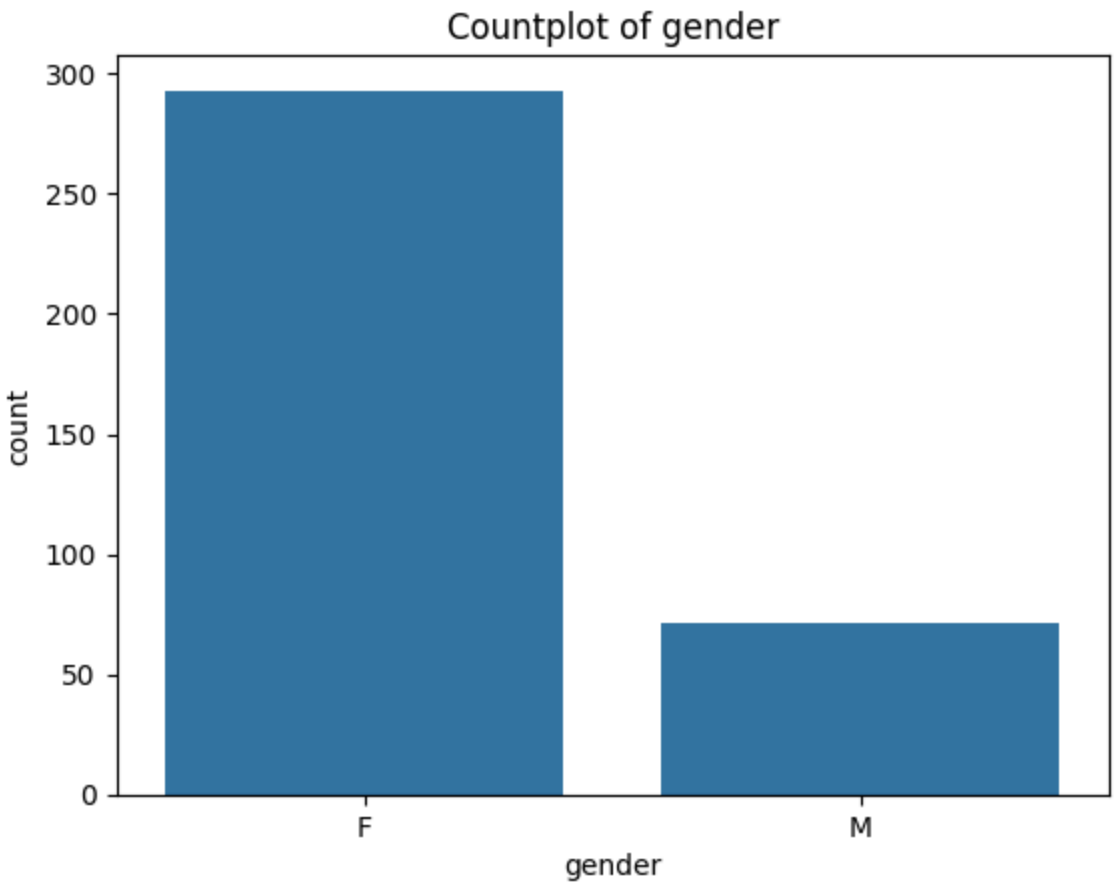
	0
age	int64
gender	object
smoking	object
hx_smoking	object
hx_radiothreapy	object
thyroid_function	object
physical_examination	object
adenopathy	object
pathology	object
focality	object
risk	object
t	object
n	object
m	object
stage	object
response	object
recurred	object

dtype: object

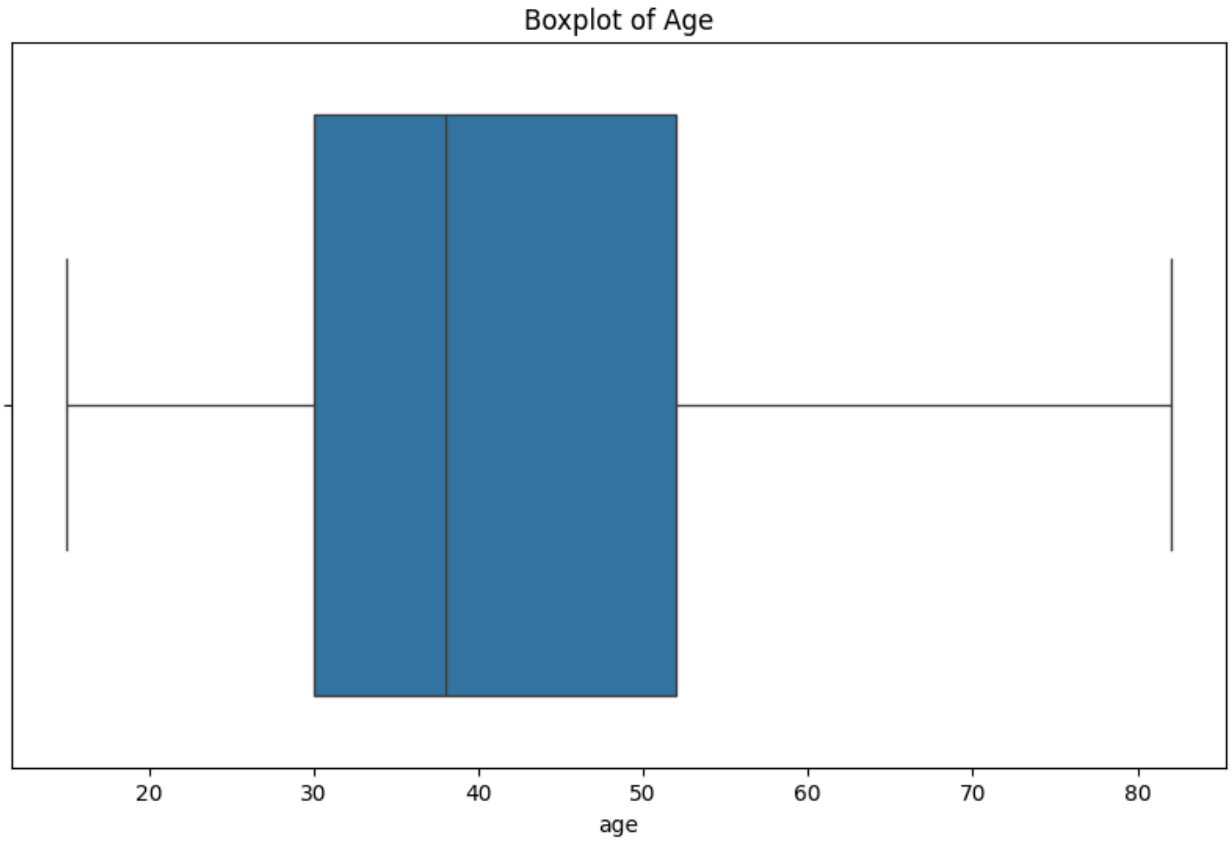
In [ ]:
```
# STEP 10: UNIVARIATE VISUALIZATIONS
# Histogram
plt.figure(figsize=(10,6))
plt.hist(df['age'])
plt.title("Histogram of Age")
plt.show()
```



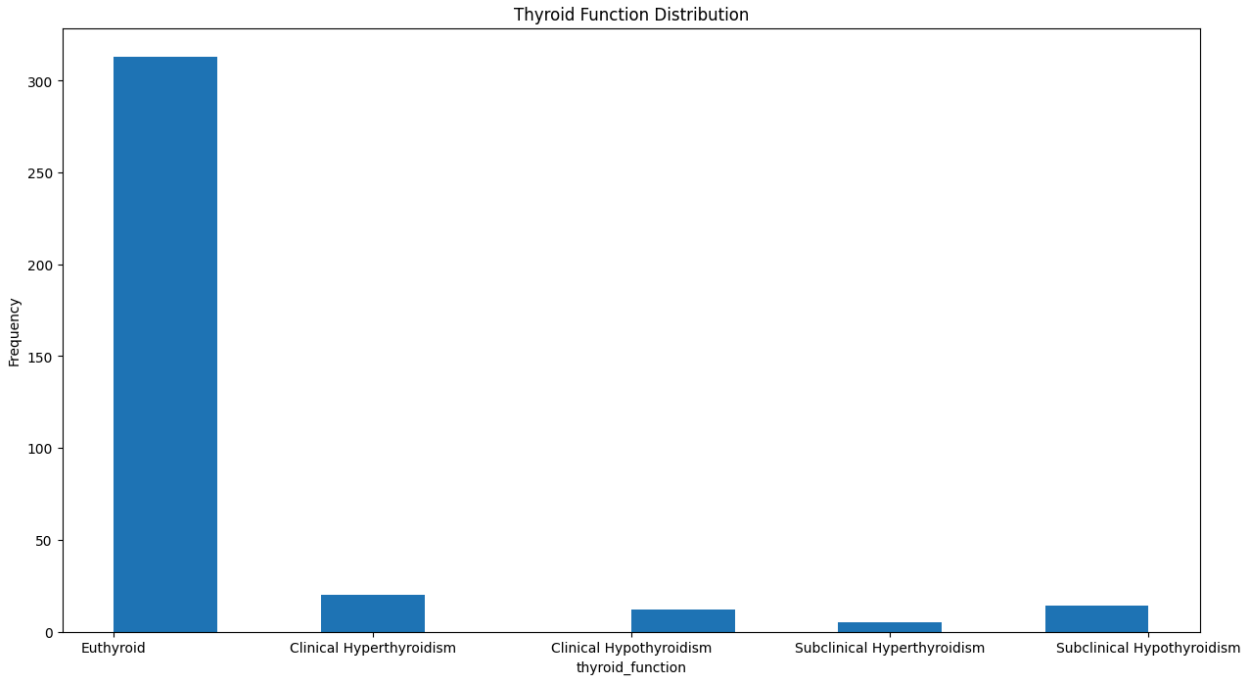
In [ ]:
```
# Countplot
plt.figure()
sns.countplot(x=df['gender'])
plt.title("Countplot of gender")
plt.show()
```



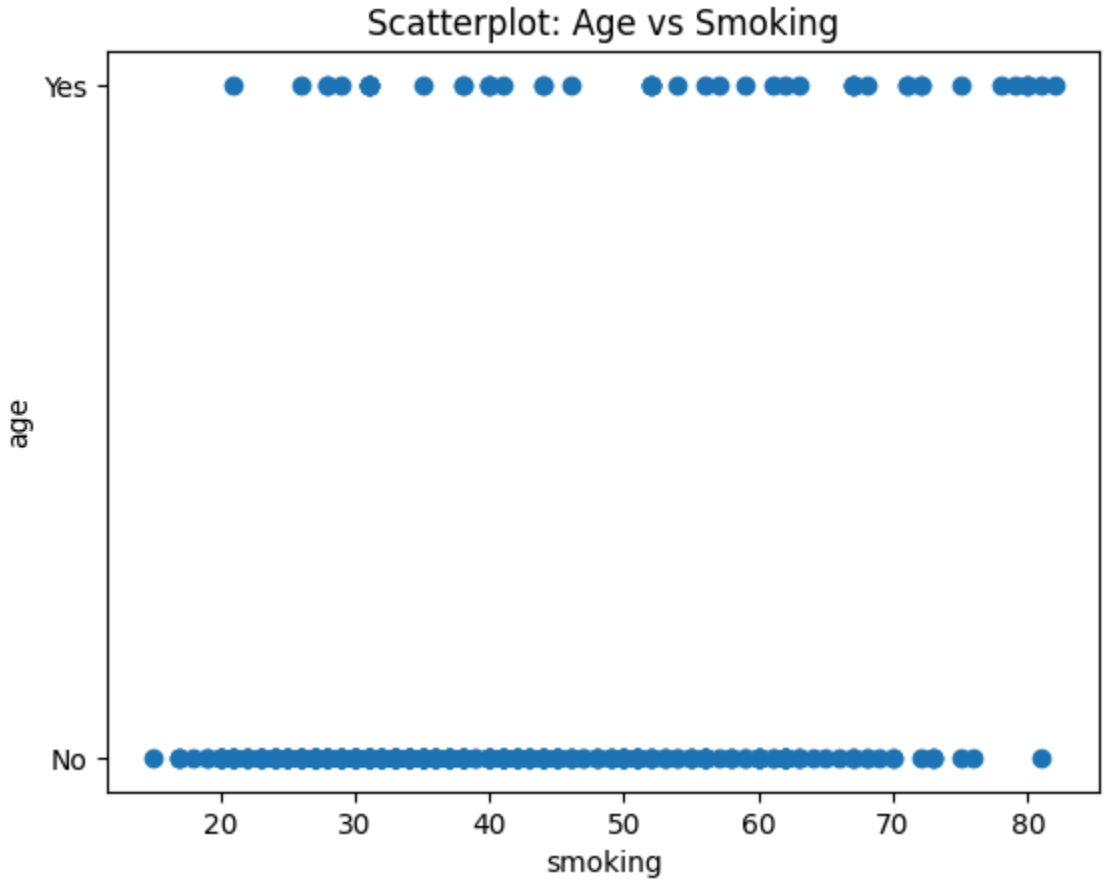
In [ ]:
```
# Boxplot
plt.figure(figsize=(10,6))
sns.boxplot(x=df['age'])
plt.title("Boxplot of Age")
plt.show()
```



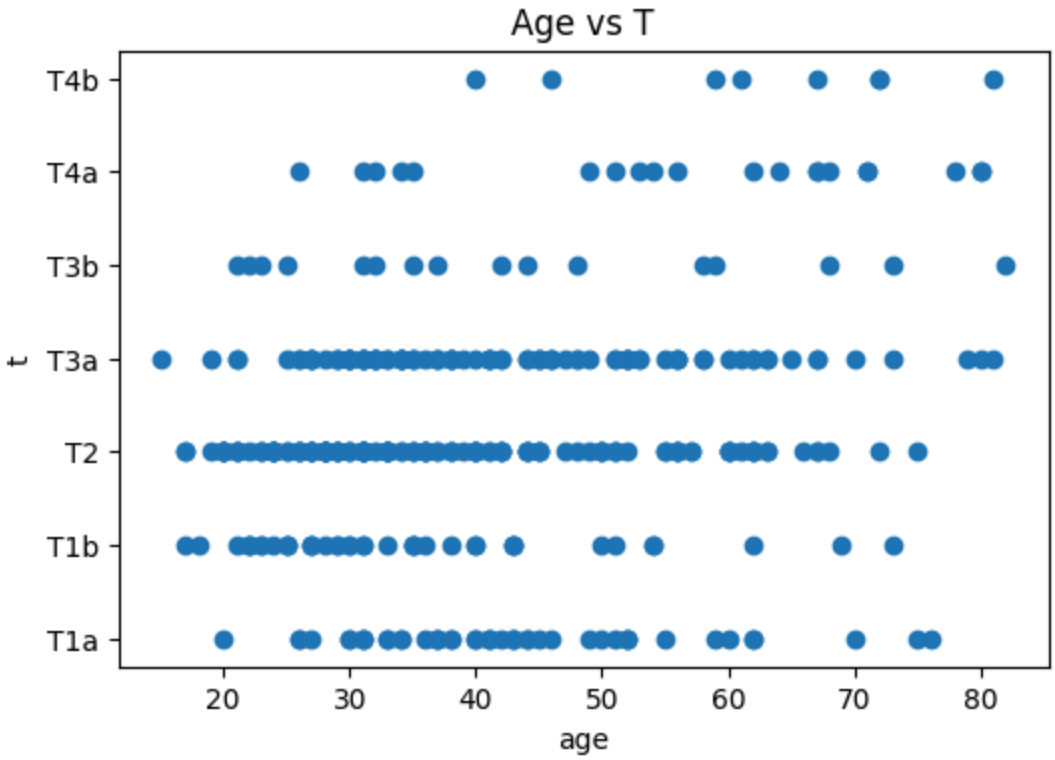
In [ ]:
```
# HISTOGRAM
plt.figure(figsize=(15,8))
plt.hist(df['thyroid_function'])
plt.title('Thyroid Function Distribution')
plt.xlabel('thyroid_function')
plt.ylabel('Frequency')
plt.show()
```



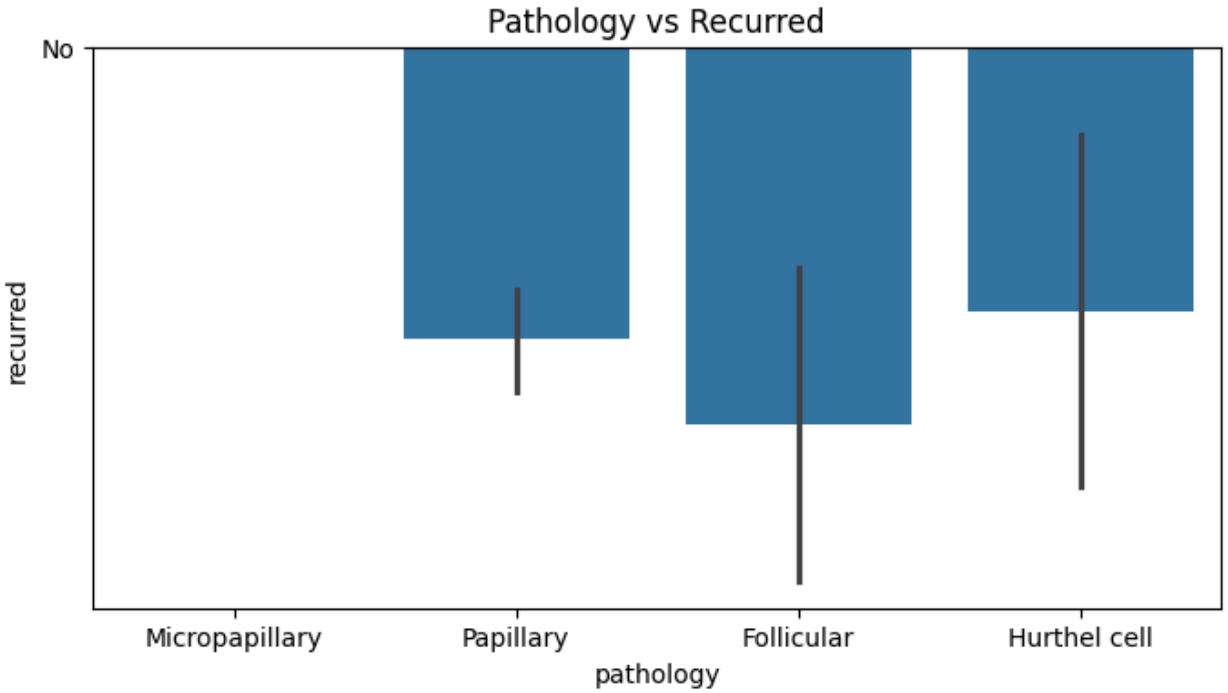
In [ ]:
```
# STEP 11: BIVARIATE VISUALIZATIONS
# SCATTER PLOT
plt.figure()
plt.scatter(df['age'], df['smoking'])
plt.title("Scatterplot: Age vs Smoking")
plt.xlabel("smoking")
plt.ylabel("age")
plt.show()
```



In [ ]:
```
# SCATTER PLOT
plt.figure(figsize=(6,4))
plt.scatter(df['age'], df['t'])
plt.title('Age vs T')
plt.xlabel('age')
plt.ylabel('t')
plt.show()
```

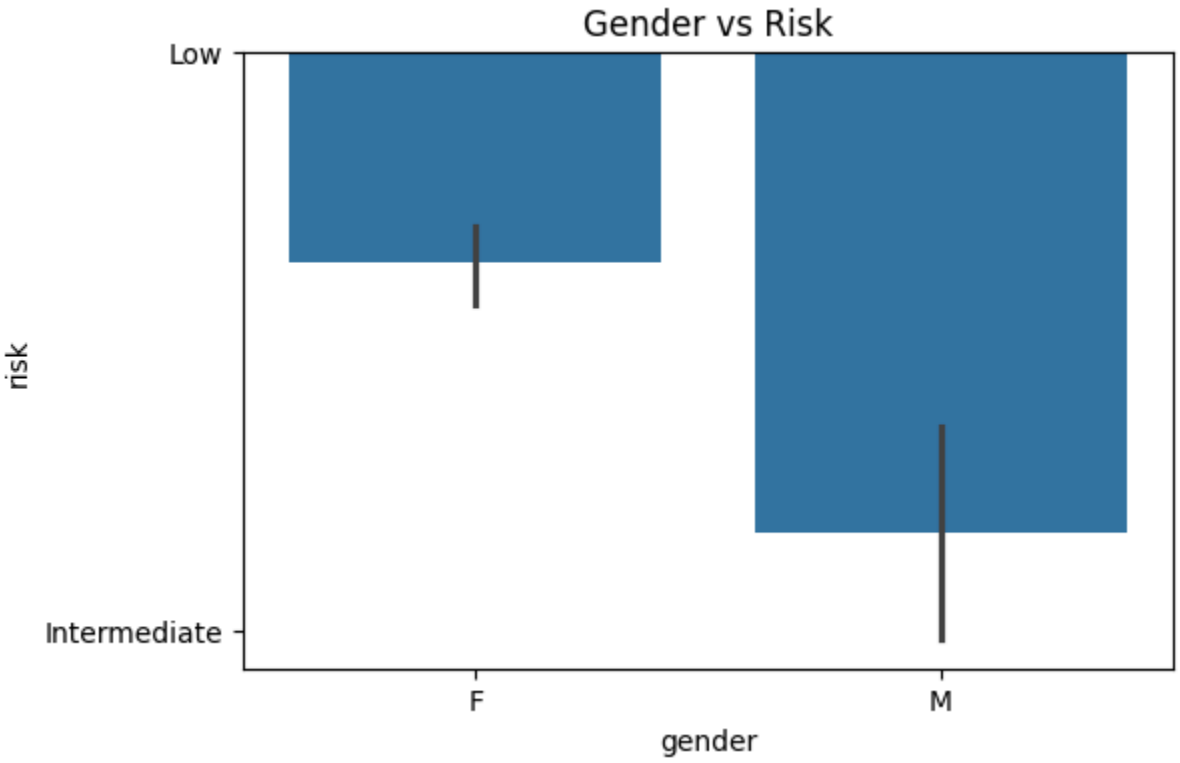


In [ ]:
```
# BARPLOT
plt.figure(figsize=(8,4))
sns.barplot(x='pathology', y='recurred', data=df)
plt.title('Pathology vs Recurred')
plt.show()
```



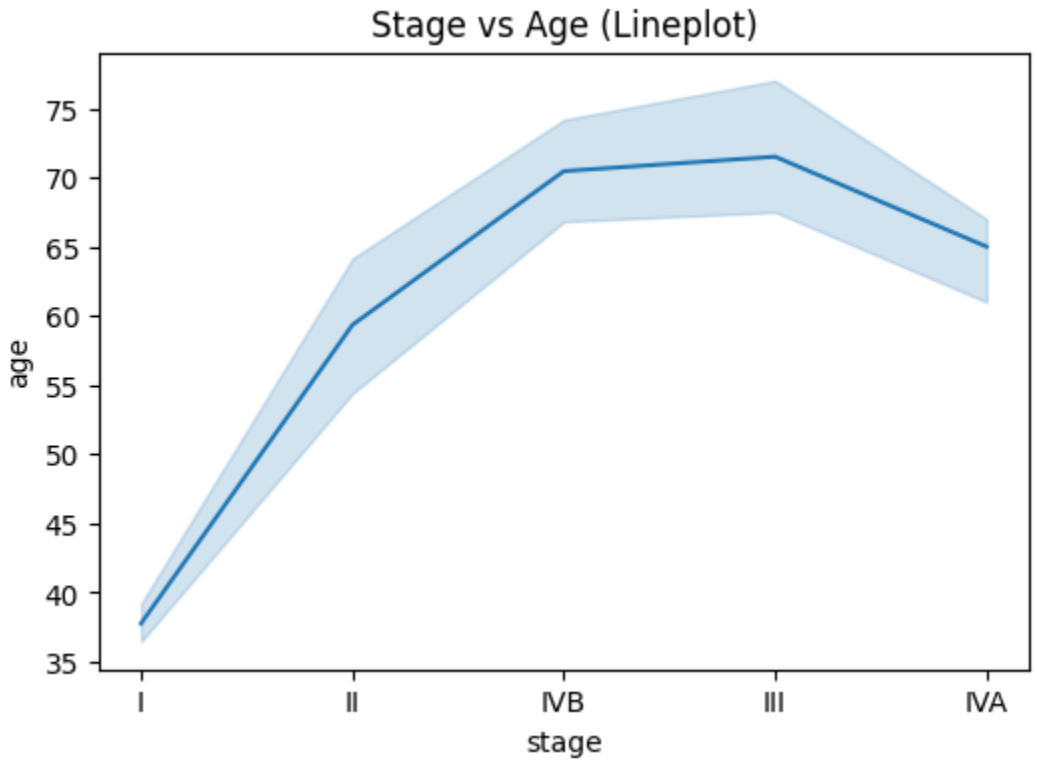
In [ ]:
```
# BARPLOT
plt.figure(figsize=(6,4))
```

```
sns.barplot(x='gender', y='risk', data=df)
plt.title('Gender vs Risk')
plt.show()
```

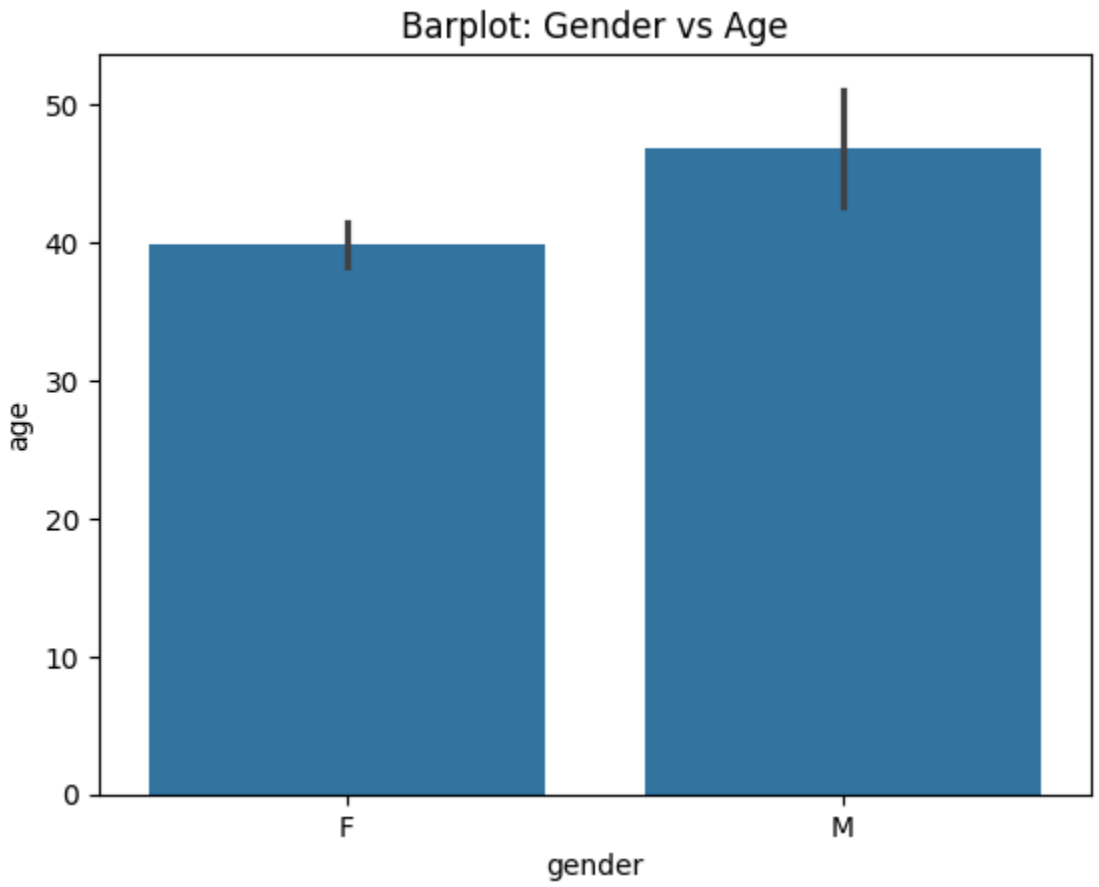


In [ ]:
```
# VIOLIN PLOT
plt.figure(figsize=(6,4))
sns.violinplot(x='stage', y='age', data=df)
plt.title('Stage vs Age (Violin Plot)')
plt.show()
```

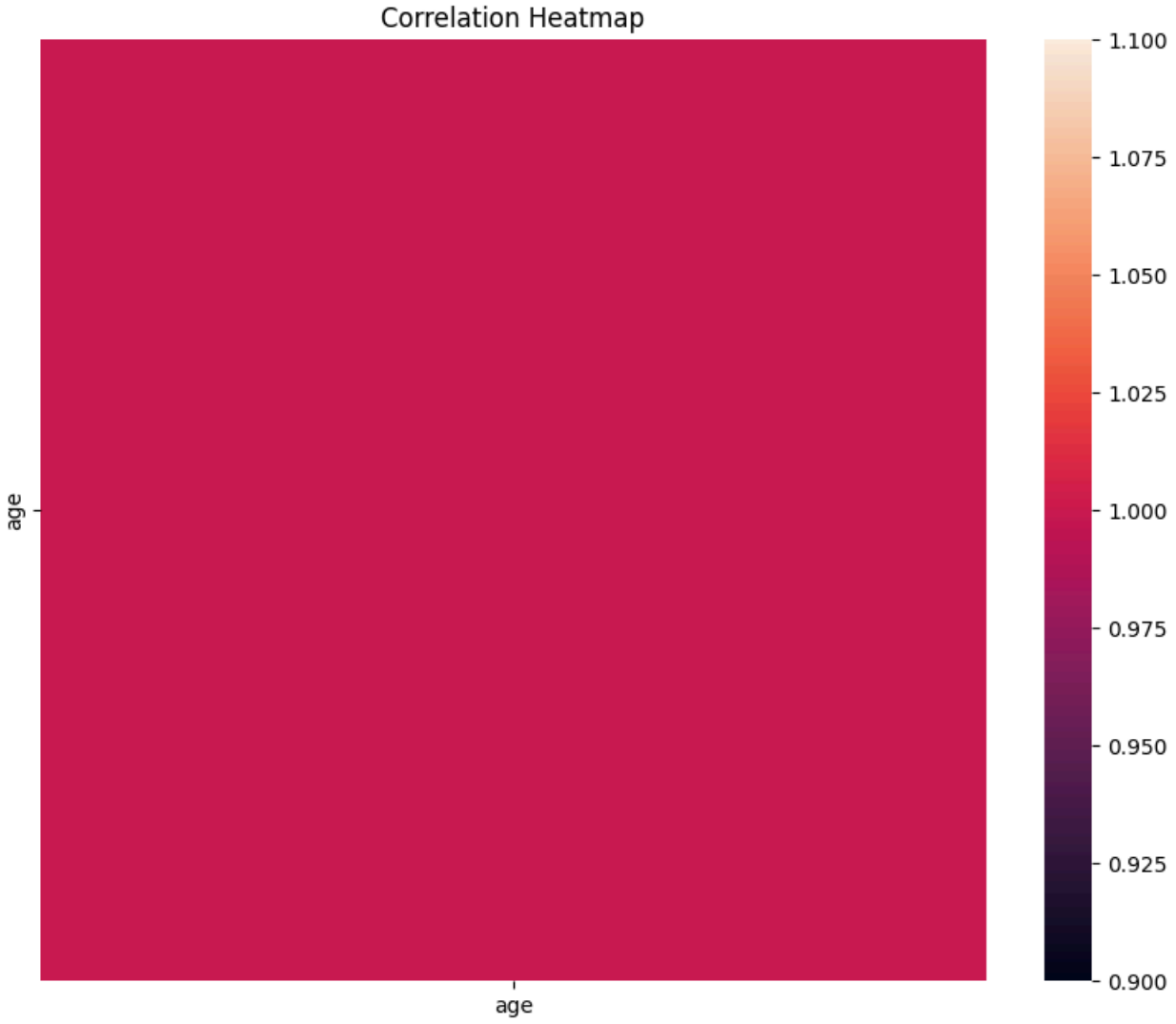
In [ ]:
```
# LINE PLOT
plt.figure(figsize=(6,4))
sns.lineplot(x='stage', y='age', data=df)
plt.title('Stage vs Age (Lineplot)')
plt.show()
```



In [ ]:
```
# BARPLOT
plt.figure()
sns.barplot(x=df['gender'], y=df['age'])
plt.title("Barplot: Gender vs Age")
plt.show()
```



In [ ]:
```
#Heatmap:  Correlation of Numeric Columns
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(numeric_only=True), annot=False)
plt.title("Correlation Heatmap")
plt.show()
```



In [ ]:
```
df.columns
```

Out[ ]:
```
Index(['age', 'gender', 'smoking', 'hx_smoking', 'hx_radiothreapy',
       'thyroid_function', 'physical_examination', 'adenopathy', 'pathology',
       'focality', 'risk', 't', 'n', 'm', 'stage', 'response', 'recurred'],
      dtype='object')
```

In [ ]:
```
# STEP 12: UNIQUE VALUES AND VALUE COUNTS OF CATEGIORICAL DATA
cat_cols = [
    'gender', 'smoking', 'hx_smoking', 'thyroid_function',
    'physical_examination', 'adenopathy', 'pathology', 'focality', 't', 'n',
    'm', 'stage', 'response', 'recurred' ]

for col in cat_cols:
    print(col)
    print(df[col].unique())
    print(df[col].value_counts())
```

```
gender
['F' 'M']
gender
F    293
M     71
Name: count, dtype: int64
smoking
['No' 'Yes']
smoking
No     315
Yes     49
Name: count, dtype: int64
hx_smoking
['No' 'Yes']
hx_smoking
No     336
Yes     28
Name: count, dtype: int64
thyroid_function
['Euthyroid' 'Clinical Hyperthyroidism' 'Clinical Hypothyroidism'
 'Subclinical Hyperthyroidism' 'Subclinical Hypothyroidism']
thyroid_function
Euthyroid                       313
Clinical Hyperthyroidism         20
Subclinical Hypothyroidism       14
Clinical Hypothyroidism          12
Subclinical Hyperthyroidism       5
Name: count, dtype: int64
physical_examination
['Single nodular goiter-left' 'Multinodular goiter'
 'Single nodular goiter-right' 'Normal' 'Diffuse goiter']
physical_examination
Multinodular goiter             135
Single nodular goiter-right     127
Single nodular goiter-left       88
Normal                            7
Diffuse goiter                    7
Name: count, dtype: int64
adenopathy
['No' 'Right' 'Extensive' 'Left' 'Bilateral' 'Posterior']
adenopathy
No           258
Right         48
Bilateral     32
Left          17
Extensive      7
Posterior      2
Name: count, dtype: int64
pathology
['Micropapillary' 'Papillary' 'Follicular' 'Hurthel cell']
pathology
Papillary         271
Micropapillary     45
Follicular         28
```

```
Hurthel cell       20
Name: count, dtype: int64
focality
['Uni-Focal' 'Multi-Focal']
focality
Uni-Focal      228
Multi-Focal    136
Name: count, dtype: int64
t
['T1a' 'T1b' 'T2' 'T3a' 'T3b' 'T4a' 'T4b']
t
T2     138
T3a     96
T1a     46
T1b     40
T4a     20
T3b     16
T4b      8
Name: count, dtype: int64
n
['N0' 'N1b' 'N1a']
n
N0     249
N1b     93
N1a     22
Name: count, dtype: int64
m
['M0' 'M1']
m
M0    346
M1     18
Name: count, dtype: int64
stage
['I' 'II' 'IVB' 'III' 'IVA']
stage
I      314
II      32
IVB     11
III      4
IVA      3
Name: count, dtype: int64
response
['Indeterminate' 'Excellent' 'Structural Incomplete'
 'Biochemical Incomplete']
response
Excellent                 189
Structural Incomplete      91
Indeterminate              61
Biochemical Incomplete     23
Name: count, dtype: int64
recurred
['No' 'Yes']
recurred
No     256
```

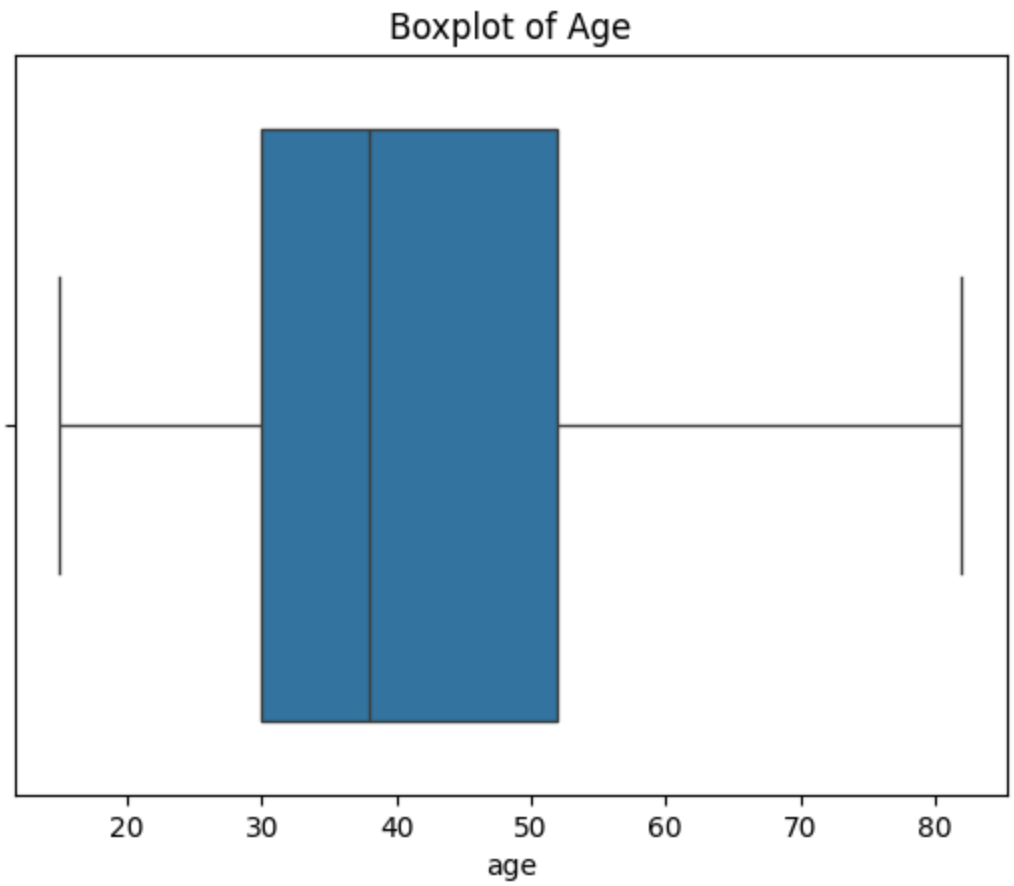
```
Yes    108
Name: count, dtype: int64
```

In [ ]:
```python
# UNIQUE VALUES AND VALUE COUNTS OF NUMERIC DATA
num_cols = ['age']
for col in num_cols:
    print(col)
    print(df[col].unique())
    print(df[col].value_counts())
```

```
age
[27 34 30 62 52 41 46 51 40 75 59 49 50 76 42 44 43 36 70 60 33 26 37 55
 31 45 20 38 29 25 21 23 24 35 54 22 69 28 17 73 18 39 57 66 32 47 56 63
 19 67 72 61 68 48 81 53 58 80 79 65 15 82 71 64 78]
age
31    21
27    13
30    12
33    12
40    11
      ..
79     1
15     1
82     1
64     1
78     1
Name: count, Length: 65, dtype: int64
```

In [ ]:
```python
# Step 13: Handle Outliers (IQR method)
for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    df = df[(df[col] >= lower) & (df[col] <= upper)]
```

In [ ]:
```python
# BOX PLOT
plt.figure()
sns.boxplot(x=df['age'])
plt.title("Boxplot of Age")
plt.show()
```



In [ ]:
```
# STEP 14: RE CHECK
df.head()
```

Out[ ]:

	age	gender	smoking	hx_smoking	hx_radiothreapy	thyroid_function	physi
0	27	F	No	No	No	Euthyroid	Sing
1	34	F	No	Yes	No	Euthyroid	M
2	30	F	No	No	No	Euthyroid	Sing
3	62	F	No	No	No	Euthyroid	Sing
4	62	F	No	No	No	Euthyroid	M

STEP 2 : MODELING

In [ ]:
```
# ALL COLUMN NAMES
df.columns
```

Out[ ]:
```
Index(['age', 'gender', 'smoking', 'hx_smoking', 'hx_radiothreapy',
       'thyroid_function', 'physical_examination', 'adenopathy', 'pathology',
       'focality', 'risk', 't', 'n', 'm', 'stage', 'response', 'recurred'],
      dtype='object')
```

In [ ]:
```
# FEATURE SELECTION AND TARGET COLUMN
x = df[['age', 'gender', 'smoking', 'hx_smoking', 'hx_radiothreapy',
       'thyroid_function', 'physical_examination', 'adenopathy', 'pathology',
       'focality', 't', 'n', 'm', 'stage', 'response', 'recurred']]
y = df['risk']
```

In [ ]:
```
# Convert Categorical Columns to Numeric
x = pd.get_dummies(x, drop_first=True)
```

In [ ]:
```
#Train-Test Split
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)
```

In [ ]:
```
#Scaling (StandardScaler)
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
```

In [ ]:
```
# Logistic Regression
# Logistic Regression predicts the probability of a class using a logistic (si
lr = LogisticRegression()
lr.fit(x_train, y_train)
y_pred_lr = lr.predict(x_test)
print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_lr))
```

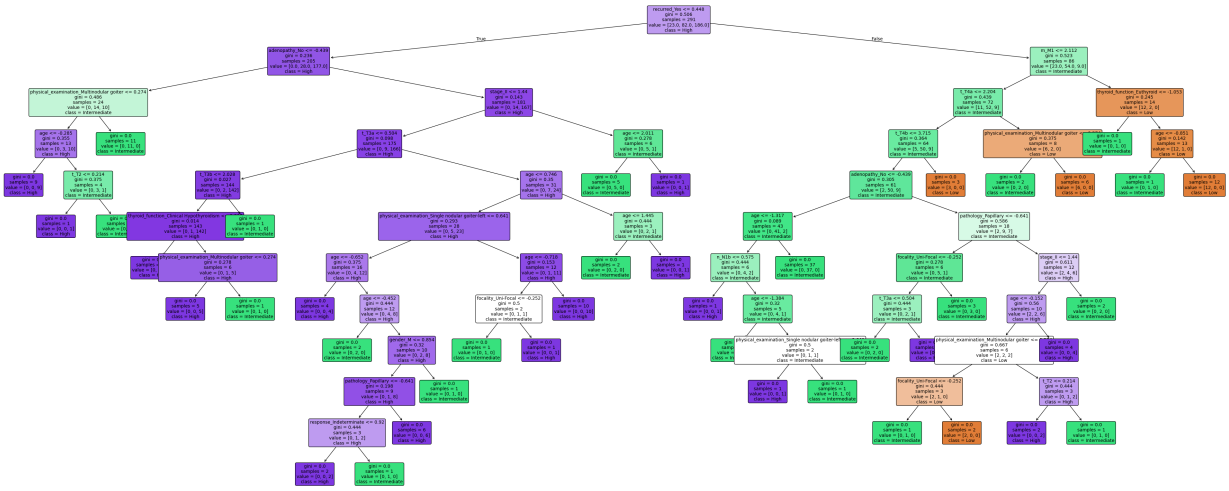
```
Logistic Regression Accuracy: 0.8493150684931506
```

In [ ]:
```
# Decision Tree
# Decision Tree splits the data based on conditions to classify samples into c
dt = DecisionTreeClassifier()
dt.fit(x_train, y_train)
y_pred_dt = dt.predict(x_test)
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
```

```
Decision Tree Accuracy: 0.821917808219178
```

In [ ]:
```
# plotting of decision tree
from sklearn.tree import plot_tree

plt.figure(figsize=(50,20))
plot_tree(
    dt,
    feature_names=x.columns,
    class_names=['Low', 'Intermediate', 'High'],
    filled=True,
    rounded=True,
    fontsize=10
)
```

```
plt.show()
```



In [ ]:
```
# Random Forest
# Random Forest combines many decision trees and takes the majority vote for b
rf = RandomForestClassifier()
rf.fit(x_train, y_train)
y_pred_rf = rf.predict(x_test)
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
```

```
Random Forest Accuracy: 0.821917808219178
```

In [ ]:
```
# KNN (K-Nearest Neighbors)
# KNN classifies a data point based on the classes of its closest neighboring
knn = KNeighborsClassifier()
knn.fit(x_train, y_train)
y_pred_knn = knn.predict(x_test)
print("KNN Accuracy:", accuracy_score(y_test, y_pred_knn))
```

```
KNN Accuracy: 0.8767123287671232
```

In [ ]:
```
# SVC (Support Vector Classifier)
# SVC finds the best boundary (hyperplane) that separates different classes.
svc = SVC()
svc.fit(x_train, y_train)
y_pred_svc = svc.predict(x_test)
print("SVC Accuracy:", accuracy_score(y_test, y_pred_svc))
```

```
SVC Accuracy: 0.863013698630137
```

In [ ]:
```
# Define base learners
clf_lr = LogisticRegression()
clf_dt = DecisionTreeClassifier(random_state=42)
clf_rf = RandomForestClassifier(n_estimators=100, random_state=42)
clf_knn = KNeighborsClassifier()
clf_svc = SVC()
```

In [ ]:
```
# Voting Classifier (Hard voting)
# Voting Classifier combines predictions of multiple models and chooses the ma
voting = VotingClassifier(
```

```
    estimators=[('lr', clf_lr), ('dt', clf_dt), ('rf', clf_rf)],
    voting='hard'
)
voting.fit(x_train, y_train)
pred_voting = voting.predict(x_test)
print("\nVoting Classifier Accuracy:", accuracy_score(y_test, pred_voting))
```

```
Voting Classifier Accuracy: 0.8356164383561644
```

In [ ]:
```
# Stacking Classifier
# Stacking trains several base models and then a final model on their combined
stack = StackingClassifier(
    estimators=[('rf', clf_rf), ('dt', clf_dt), ('knn', clf_knn)],
    final_estimator=LogisticRegression(),
    cv=5
)
stack.fit(x_train, y_train)
pred_stack = stack.predict(x_test)
print("\nStacking Classifier Accuracy:", accuracy_score(y_test, pred_stack))
```

```
Stacking Classifier Accuracy: 0.8493150684931506
```

In [ ]:
```
# Gradient Boosting Classifier
# Gradient Boosting builds models step-by-step, each fixing the errors of the
gb = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, max_depth
gb.fit(x_train, y_train)
pred_gb = gb.predict(x_test)
print("\nGradient Boosting Accuracy:", accuracy_score(y_test, pred_gb))
```

```
Gradient Boosting Accuracy: 0.8493150684931506
```

In [ ]:
```
# AdaBoost Classifier
# AdaBoost gives more weight to wrongly classified points and improves the mod
ada = AdaBoostClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
ada.fit(x_train, y_train)
pred_ada = ada.predict(x_test)
print("\nAdaBoost Accuracy:", accuracy_score(y_test, pred_ada))
```

```
AdaBoost Accuracy: 0.7671232876712328
```

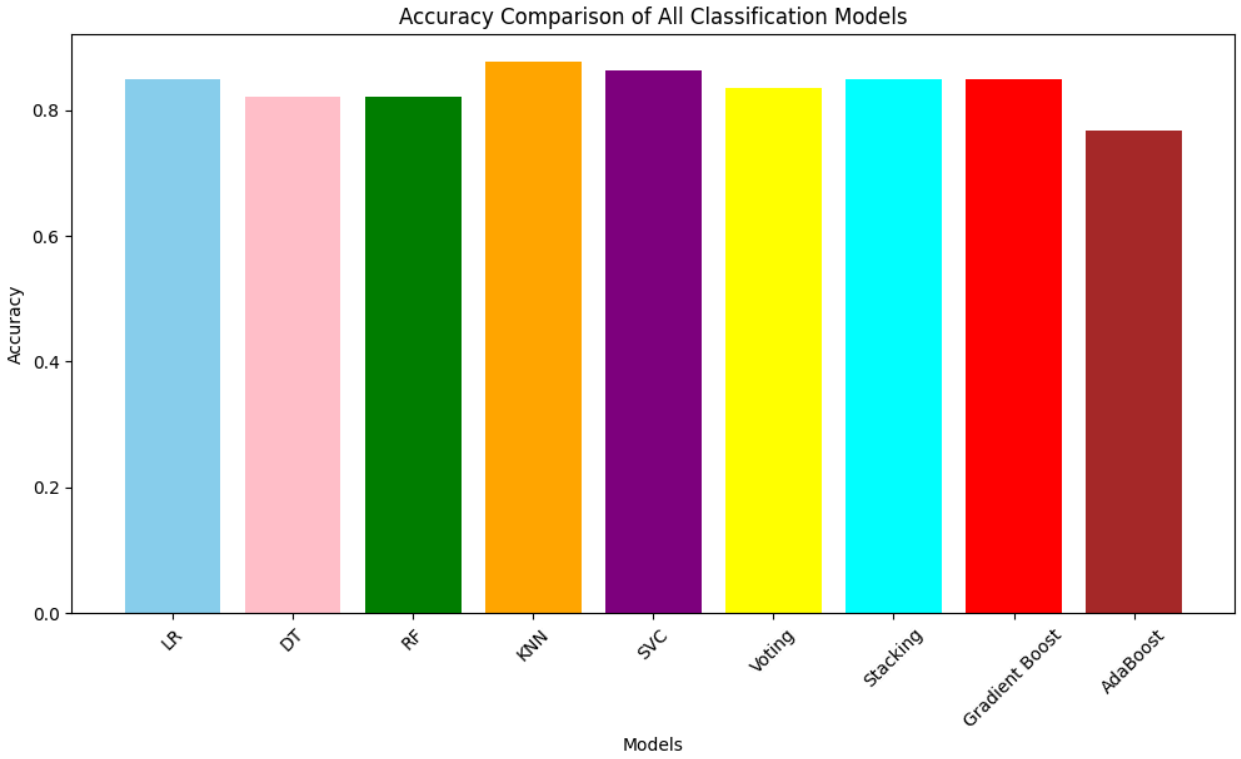
In [ ]:
```
from sklearn.metrics import accuracy_score

LR_accuracy = accuracy_score(y_test, y_pred_lr)
DT_accuracy = accuracy_score(y_test, y_pred_dt)
RF_accuracy = accuracy_score(y_test, y_pred_rf)
KNN_accuracy = accuracy_score(y_test, y_pred_knn)
SVC_accuracy = accuracy_score(y_test, y_pred_svc)
Voting_accuracy = accuracy_score(y_test, pred_voting)
Stacking_accuracy = accuracy_score(y_test, pred_stack)
GB_accuracy = accuracy_score(y_test, pred_gb)
AdaBoost_accuracy = accuracy_score(y_test, pred_ada)

print(" MODEL ACCURACY SUMMARY ")

print("Logistic Regression Accuracy:         ", LR_accuracy)
```

```
print("Decision Tree Accuracy:                ", DT_accuracy)
print("Random Forest Accuracy:                ", RF_accuracy)
print("KNN Accuracy:                          ", KNN_accuracy)
print("SVC Accuracy:                          ", SVC_accuracy)

print("Voting Classifier Accuracy:            ", Voting_accuracy)
print("Stacking Classifier Accuracy:          ", Stacking_accuracy)
print("Gradient Boosting Accuracy:            ", GB_accuracy)
print("AdaBoost Accuracy:                     ", AdaBoost_accuracy)
```

```
 MODEL ACCURACY SUMMARY
Logistic Regression Accuracy:          0.8493150684931506
Decision Tree Accuracy:                0.821917808219178
Random Forest Accuracy:                0.821917808219178
KNN Accuracy:                          0.8767123287671232
SVC Accuracy:                          0.863013698630137
Voting Classifier Accuracy:            0.8356164383561644
Stacking Classifier Accuracy:          0.8493150684931506
Gradient Boosting Accuracy:            0.8493150684931506
AdaBoost Accuracy:                     0.7671232876712328
```

In [2]:

```
# STEP 11: BAR GRAPH FOR MODEL ACCURACY COMPARISON

models = ['LR', 'DT', 'RF', 'KNN', 'SVC', 'Voting', 'Stacking', 'Gradient Boos

accuracies = [
    0.8493150684931506,   # Logistic Regression
    0.821917808219178,    # Decision Tree
    0.821917808219178,    # Random Forest
    0.8767123287671232,   # KNN
    0.863013698630137,    # SVC
    0.8356164383561644,  # Voting Classifier
    0.8493150684931506,  # Stacking Classifier
    0.8493150684931506,  # Gradient Boosting
    0.7671232876712328   # AdaBoost
]

import matplotlib.pyplot as plt

colors = ['skyblue', 'pink', 'green', 'orange', 'purple', 'yellow', 'cyan', 'r

plt.figure(figsize=(12,6))
plt.bar(models, accuracies, color=colors)
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.title("Accuracy Comparison of All Classification Models")
plt.xticks(rotation=45)
plt.show()
```



From the accuracy results :

1. KNN – 0.8767 (Highest Accuracy)
2. Decision Tree - 0.8219 (Less Accuracy)

In [ ]: